청기백기 게임 업그레이드

응용 8-11-1

21개의 업그레이드 · 원본 177줄 분석 · 버그 9개 수정

이름:다바수렌 세렌톡토흐

학번:2024764050

게임프로그래밍

목차

01.

원본 소스 분석

게임 규칙 · 함수 · 실행 순서

02.

원본 실행 화면과 문제점

실제 실행 결과와 버그 9개

03.

업그레이드 21개

한 장에 하나씩, 전/후 화면 비교

04.

최종 실행 화면 · 요약표

추가 · 변경 · 삭제, 총 개수

05.

결론 / 느낀 점

청기백기 게임

최대 4명이 차례로 플레이하며, 화면의 깃발 명령에 맞는 키를 1초 안에 누르는 177줄짜리 C 게임

키 조작

	올려	내려
청기	a	z
백기	k	m

두 부분 명령(“백기 내리지말고 청기 내려!”)은 뒷부분만 따른다

실행 순서

1. 인원수(4명 이하) · 문제 수(10개 이하) · 이름 입력
2. rand()로 플레이 순서 섞기
3. 플레이어마다 규칙 화면 → 무작위 깃발 명령
4. clock()으로 반응 시간 측정
5. 정답이면서 1초 미만이면 O, 아니면 X
6. 모두 끝나면 결과표 출력

함수 구조

보조 함수: display_rule (키 안내 후 5초 대기) · gotoxy (콘솔 커서 이동)
전역 변수 없이 배열을 함수 인자로 넘기는 구조

01.

main

인원수 · 문제 수 · 이름 입력,
순서 섞기, 전체 진행

02.

game_control

플레이어 한 명의 게임:
명령 출력, 키 판정,
시간 기록
(인원수만큼 반복)

03.

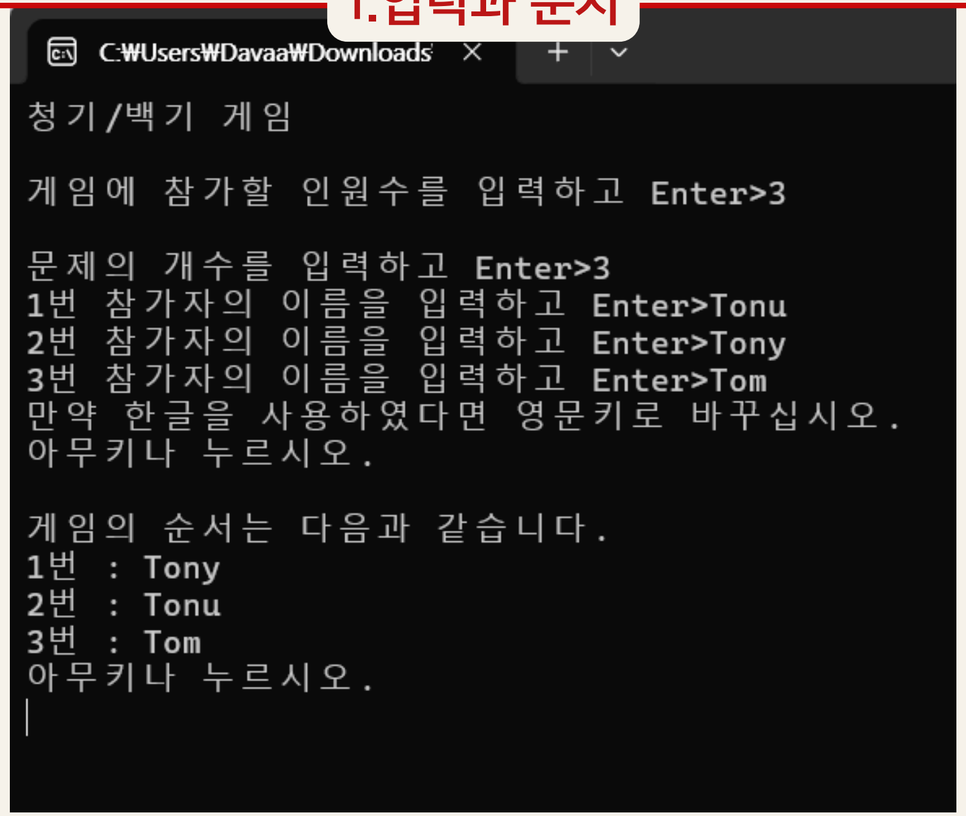
display_result

문제별 O/X와 시간,
맞은개수 · 소요시간
표 출력

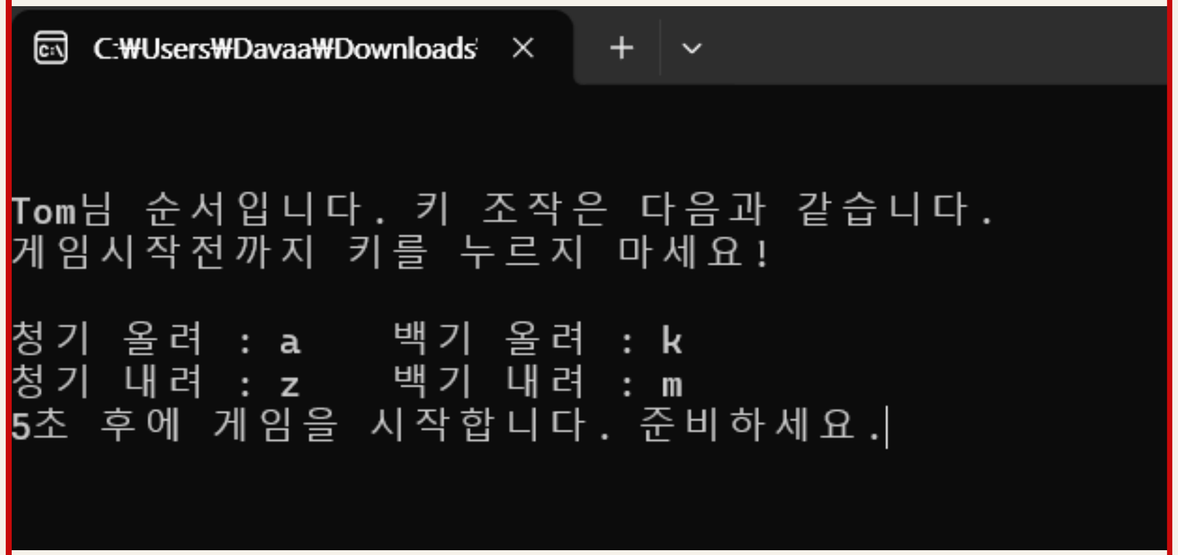
원본 실행 화면

Dev-C++에서 3명 · 3문제로 직접 실행한 결과. 문제마다 맞았는지 보이지 않고, 끝난 뒤에만 결과표가 나온다.

1.입력과 순서



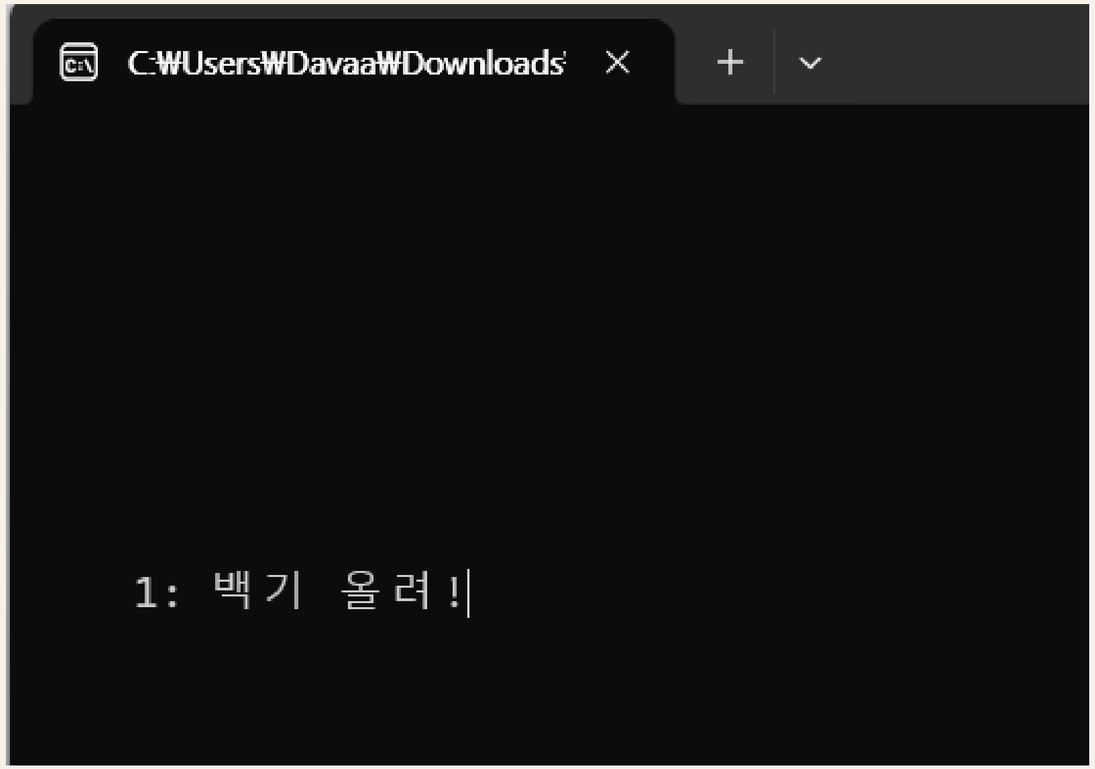
2. 규칙 안내



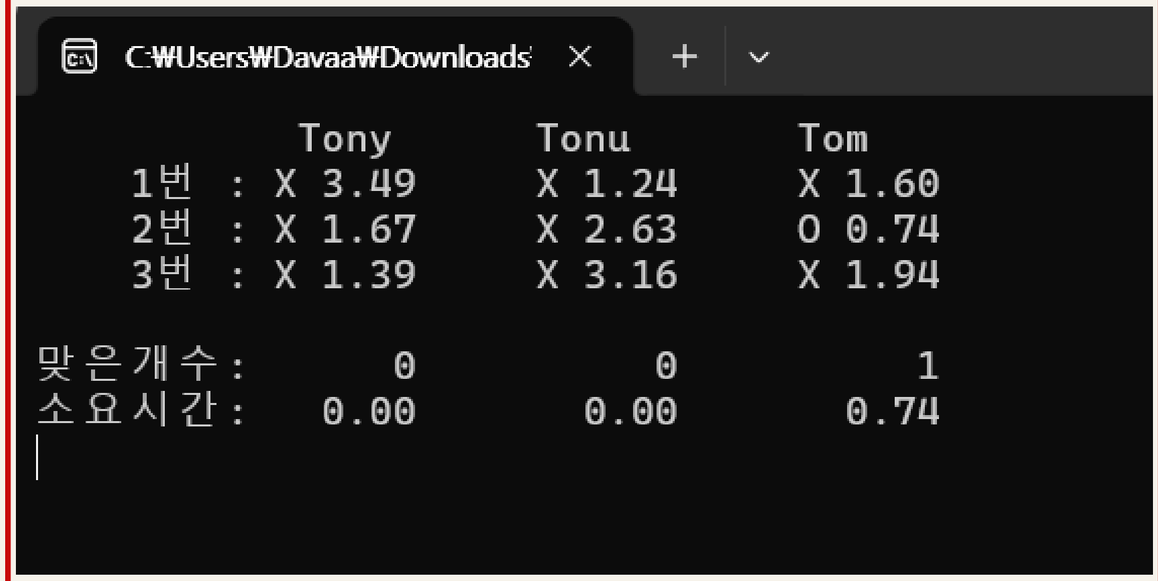
원본 실행 화면

Dev-C++에서 3명 · 3문제로 직접 실행한 결과. 문제마다 맞았는지 보이지 않고, 끝난 뒤에만 결과표가 나온다.

① 문제 출력



② 결과표



응용 8-11-1 문제점

01. **order[4], solution[4][10]**
5명·11문제를 넣어도 그대로 진행 → 배열 칸을 넘어감 | 해결: 범위 밖이면 다시 입력

02. **user_name[4][7]**
7자 이상 이름이 다음 사람 칸을 덮어씀 | 해결: 6자까지만 받기

03. **display_rule() 기다리는 중 키 입력**
기다리는 동안 누른 키가 1번 문제 답이 됨 | 해결: 남은 키 지우기

04. **key=getch();**
Caps Lock이 켜져 있으면 정답을 눌러도 X | 해결: 대문자를 소문자로 바꾸기

05. **CLK_TCK**
옛날 이름이라 다른 컴파일러에서 안 될 수 있음 | 해결: CLOCKS_PER_SEC

06. **scanf("%s", &user_name[i])**
배열 이름 앞에 & 가 필요 없음 (경고 발생) | 해결: & 지우기

07. **min_time**
이름은 '최소 시간'인데 실제로는 합계 | 해결: total_time으로 이름 바꾸기

08. **순서 섞기 반복문**
어떤 순서는 더 자주 나옴 (불공평) | 해결: Fisher-Yates 섞기

09. **key=getch(); 무한 대기**
키를 안 누르면 시간이 지나도 계속 기다림 | 해결: 시간이 지나면 자동으로 넘어가기

UPGRADE 01 / 21

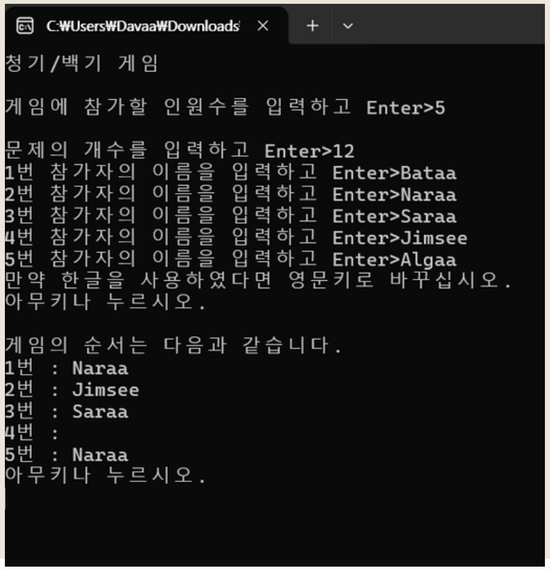
인원수 · 문제 수 입력 범위 검사

추가

X 인원수 5, 문제 12를 넣어도 그대로 진행된다 → 4칸짜리 배열 밖에 값을 쓰게 된다

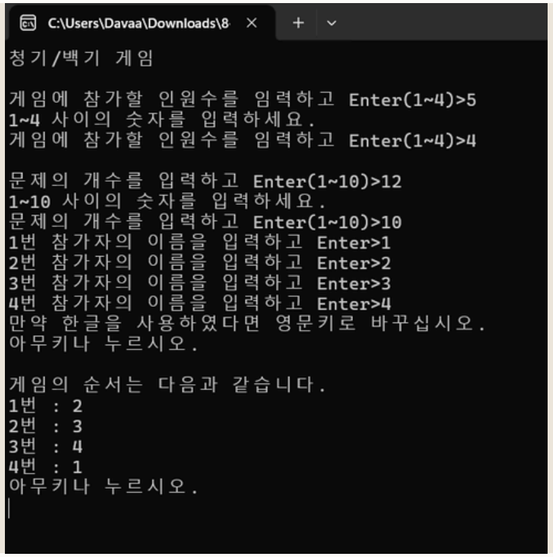
O 범위 밖 숫자를 넣으면 다시 입력받는다

X 원본 (Before)




```
scanf("%d", &user_number);   /* 인원수: 범위 검사 없음 (5 입력 가능) */
...
scanf("%d", &quest_number);  /* 문제 수: 범위 검사 없음 (12 입력 가능) */
/* → order[4], solution[4][10] 배열 밖에 값을 씀 → 게임 오류·종료 */
```

O 업그레이드 (After)




```
user_number=input_number(          /* 1~4 아니면 다시 입력 */
    "...인원수(1~4)...", 1, MAX_USER);
quest_number=input_number(         /* 1~10 아니면 다시 입력 */
    "...문제의 개수(1~10)...", 1, MAX_QUEST);
```

UPGRADE 02 / 21

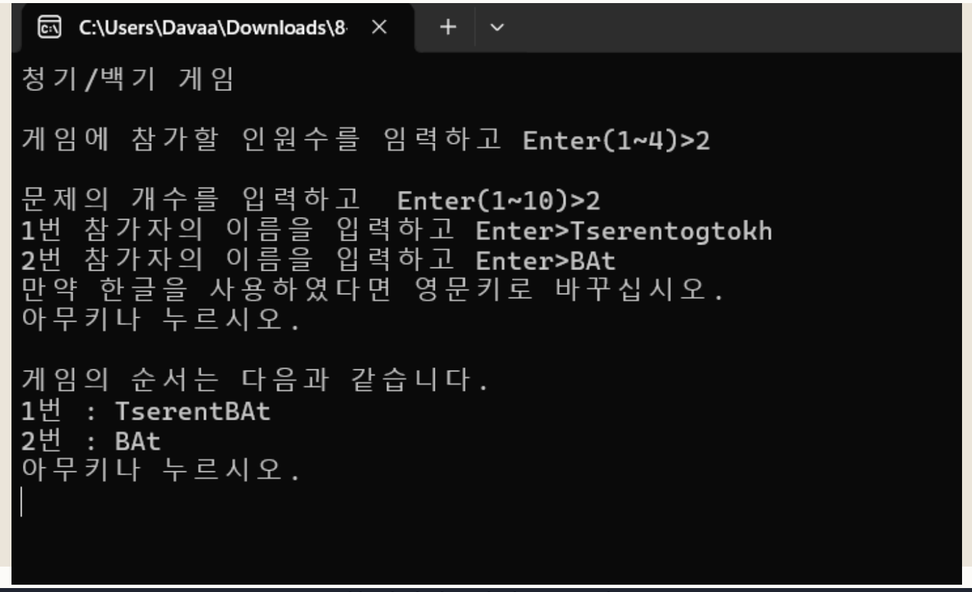
이름 길이 제한

변경

X 이름은 6글자까지인데 제한이 없다 → 'ABCDEFGHIJ'가
다음 사람 이름을 덮어 'ABCDEFGBat'로 나온다

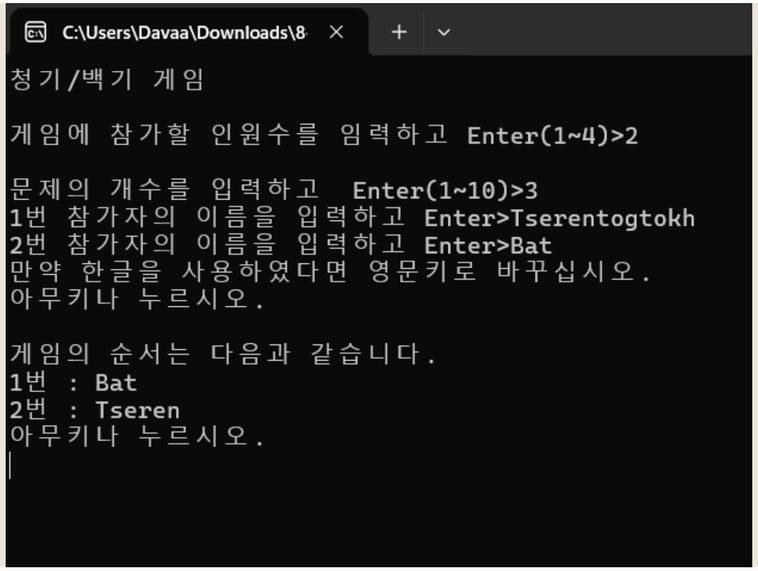
O 6글자까지만 받고 남은 글자는 버린다

X 원본 (Before)




```
char user_name[4][7];          /* 한 사람당 7바이트 = 6자 + '\0' */
scanf("%s", &user_name[i]);    /* 길이 제한 없음 → 'ABCDEFGHIJ' 입력 가능 */
/* → 다음 사람 칸까지 덮어씀: 'ABCDEFG' + 'Bat' = 'ABCDEFGBat' */
```

O 업그레이드 (After)




```
scanf("%6s", user_name[i]);    /* 최대 6자만 읽음 */
clear_input();                 /* 남은 'GHIJ'는 버림 → 다음 이름에 안 섞임 */
```

UPGRADE 03 / 21

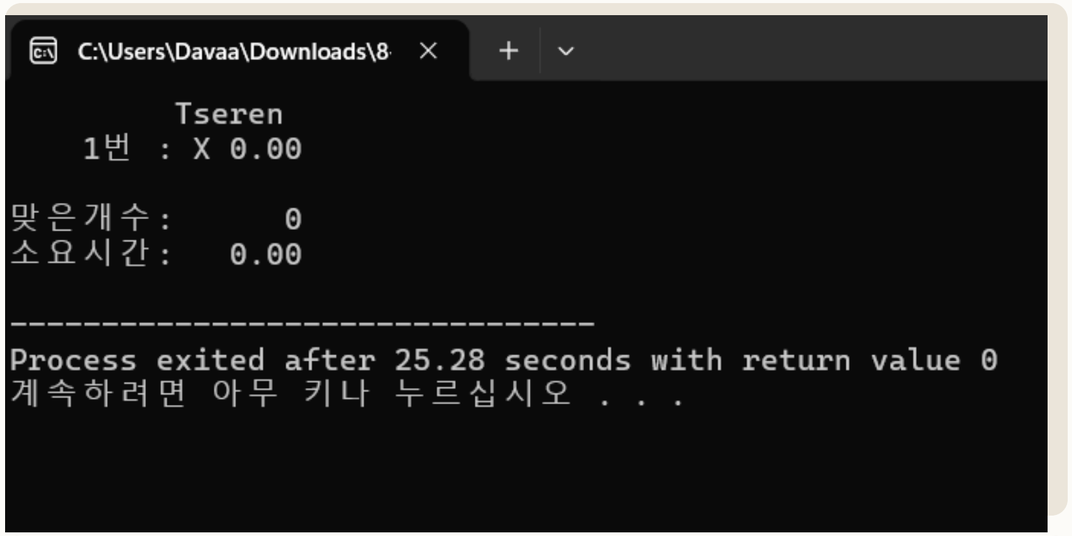
미리 누른 키 무시

추가

X 규칙 화면(7초) 동안 누른 키가 남아 있다가 1번 문제의 답이 된다 → 1번이 0.00초 X

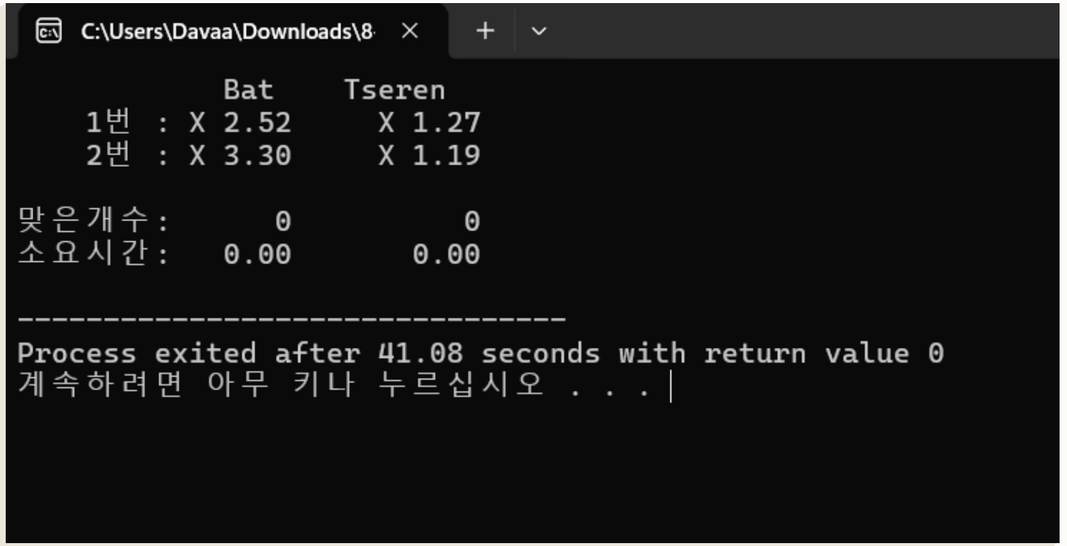
O 기다리는 동안 누른 키를 모두 지운다

X 원본 (Before)




```
display_rule();          /* 7초 동안 규칙 화면 */
for(i=0;i<quest;i++) {
    ...
    key=getch();         /* 규칙 화면 때 누른 키를 바로 읽음 → 0.00초 */
```

O 업그레이드 (After)




```
display_rule();
clear_keys();          /* while(kbhit()) getch(); → 남은 키 모두 버림 */
```

UPGRADE 04 / 21

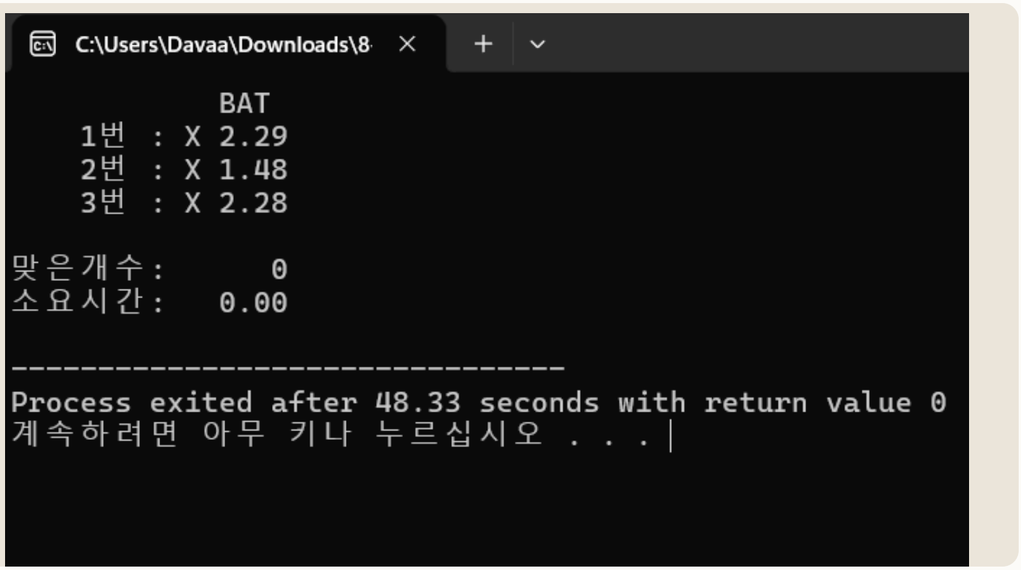
대 · 소문자 모두 인식

변경

X Caps Lock이 켜져 있으면 'A'가 들어와서 맞는 키를 눌러도 모두 X가 된다

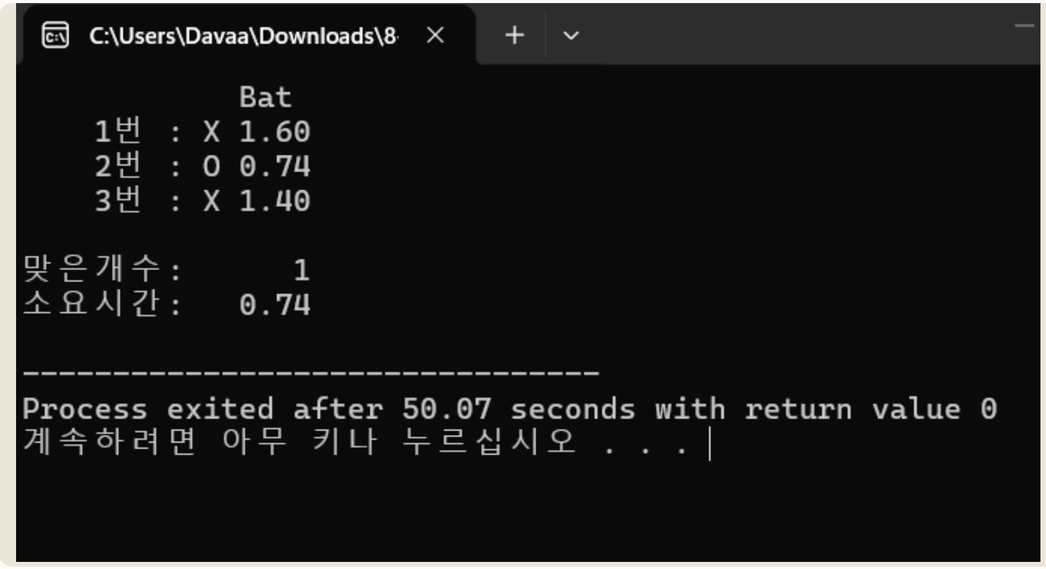
O 'A'를 'a'로 바꿔서 비교한다

X 원본 (Before)




```
key=getch();              /* Caps Lock → 'A' 가 들어옴 */
...
if (... && (key=='a'))    /* 'A' != 'a' → 정답인데 X */
```

O 업그레이드 (After)




```
key=tolower(getch());     /* 'A' → 'a' 로 바꿔서 비교 */
                          /* 아래 if 문은 그대로 */
```

UPGRADE 05 / 21

시간 계산 이름 바꾸기

변경

X CLK_TCK는 옛날 이름이라 안 되는 컴파일러가 있다

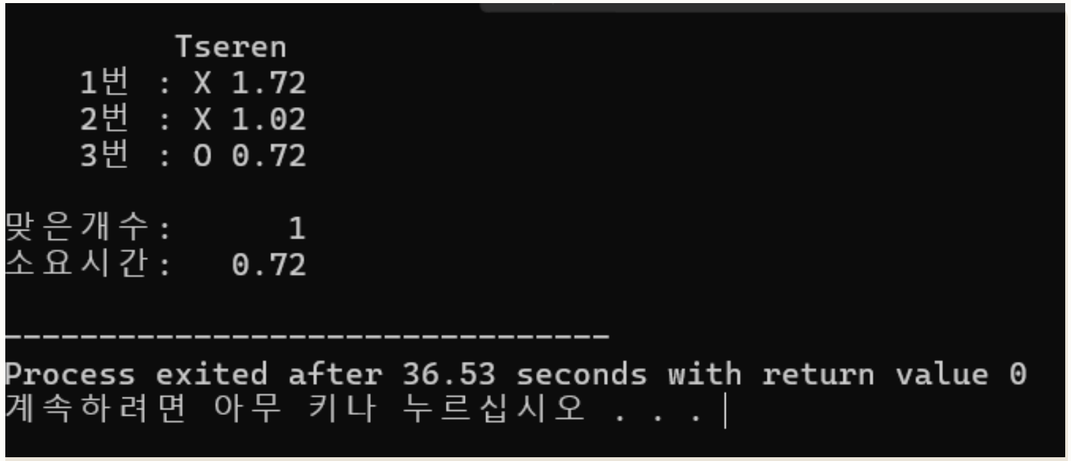
O 어디서나 되는 CLOCKS_PER_SEC로 바꾼다

X 원본 (Before)

```
check[order][i]=
    (double)(end-start)/CLK_TCK;       /* 옛날 이름 */
```

clock()은 시간을 '틱' 단위로 잰다.
틱 ÷ 1초당 틱 수 = 초
CLK_TCK는 옛날 이름이라 컴파일러에 따라 없을 수 있다

O 업그레이드 (After) · 시간 측정 그대로 동작



CLOCKS_PER_SEC는 어느 컴파일러에서나 쓸 수 있다.
값은 둘 다 1000 → 결과는 똑같다.

```
check[order][i]=
    (double)(end-start)/CLOCKS_PER_SEC; /* 표준 이름, 값은 같음(1000) */
```

필요 없는 & 지우기 · 경고 없애기

삭제

X 배열 이름에는 & 가 필요 없다 → 경고를 모두 켜고
컴파일하면 원본은 경고 9개

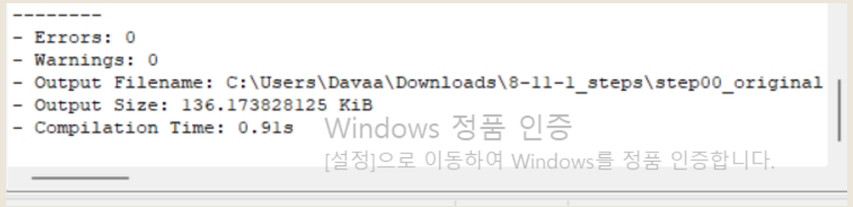
O & 지우고 const · (SHORT) 추가 → 경고 0개

X 원본 (Before)




```
scanf("%s", &user_name[i]);      /* & 필요 없음 → 경고 */
char *question[]={...};          /* 문자열 상수 → 경고 */
COORD Pos = {x - 1, y - 1};      /* int → SHORT 변환 → 경고 */
```

O 업그레이드 (After)




```
scanf("%6s", user_name[i]);                  /* & 삭제 */
const char *question[]={...};                /* const 추가 */
COORD Pos = {(SHORT)(x - 1), (SHORT)(y - 1)};  /* SHORT로 바꿈 */
```

UPGRADE 07 / 21

변수 이름 바로잡기

변경

X min_time은 '가장 짧은 시간'이 아니라 시간의 합계 →
이름과 내용이 달라 헷갈린다

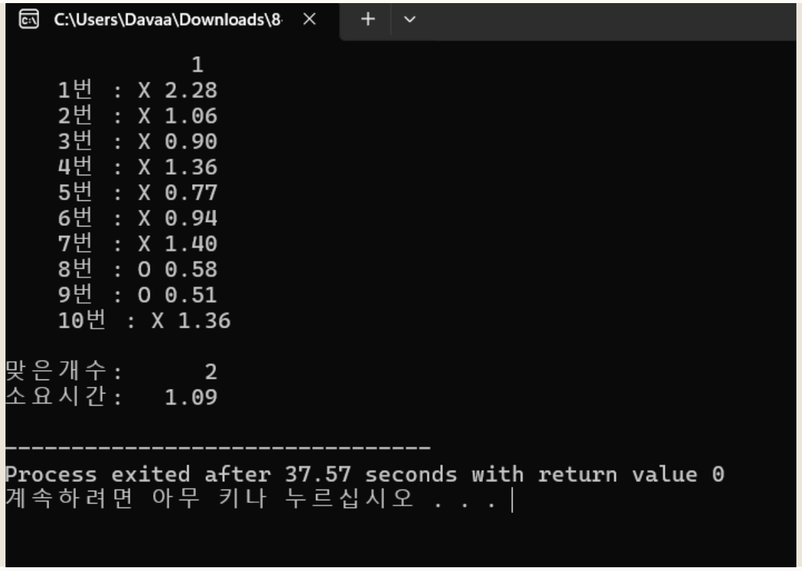
O 이름을 total_time(합계)으로 바꾼다

X 원본 (Before)

```
double min_time[4]={0};                /* 이름은 '최소' */
...
min_time[order[i]]+=check[order[i]][j]; /* 실제로는 더함 = 합계 */
```

+= 로 계속 더하므로 최소값이 아니라 합계다. 결과표의
'소요시간'(0.51+0.54+0.45=1.50)이 바로 이 합계라서 이름을
total_time 으로 바꿨다.

O 업그레이드 (After) · 소요시간 = total_time




```
double total_time[MAX_USER]={0};       /* 이름 = 내용 (합계) */
...
total_time[order[i]]+=check[order[i]][j];
```

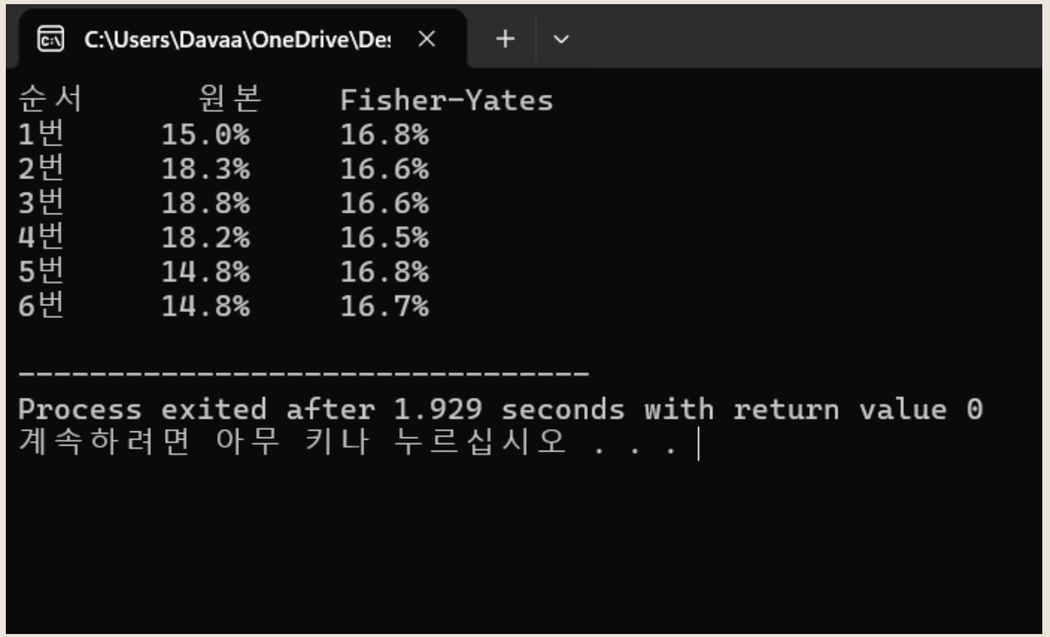
공정한 순서 섞기 (Fisher-Yates)

변경

X 원본 섞기는 순서마다 나올 확률이 다르다 → 3명 기준 14.8% ~ 18.8%

O 공평한 섞기(Fisher-Yates) → 모든 순서가 약 16.7%

60,000번 섞기 테스트 결과



섞는 코드 비교

X 원본 — 매번 0~n-1 중 아무 자리

```
for(i=0;i<user_number;i++)
{
 rnd=rand()%user_number;
 temp=order[i];
 order[i]=order[rnd];
 order[rnd]=temp;
}
```

O Fisher-Yates — 0~i 중 한 자리

```
for(i=user_number-1;i>0;i--)      /* 뒤에서부터 */
{
    rnd=rand()%(i+1);             /* 0~i 중에서만 고름 */
    temp=order[i];
    order[i]=order[rnd];
    order[rnd]=temp;
}
```

UPGRADE 09 / 21

정답 / 오답 효과음

추가

X 맞았는지 틀렸는지 소리로 알 수 없다

O 정답은 높은 소리, 오답 · 시간초과는 낮은 소리

X 원본 (Before)

```
else
    sol[order][i]=0;
}
/* 판정 후 바로 다음 문제
   소리 없음 */
```

Beep(주파수, 길이ms): 정답은 880Hz 0.12초, 오답과 시간초과는 220Hz 0.3초. 시간 측정이 끝난 뒤에 소리를 내므로 기록에는 영향이 없다.

O 업그레이드 (After)

```
if (!sol[order][i])                 /* 오답 */
    Beep(220, 300);
else if (check[order][i]>=limit[i])  /* 시간초과 */
    Beep(220, 300);
else Beep(880, 120);                /* 정답 */
```

UPGRADE 10 / 21

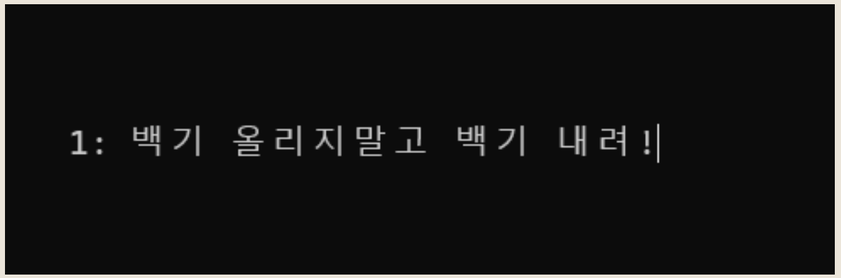
문제마다 바로 결과 보여주기

추가

X 게임이 끝난 뒤 결과표에서만 맞았는지 확인

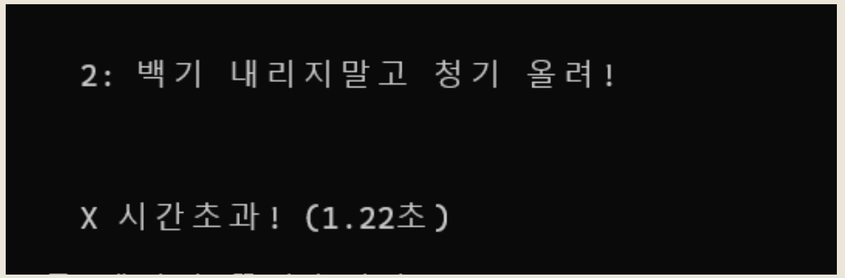
O 문제마다 O/X와 걸린 시간을 바로 표시

X 원본 (Before)




```
key=getch();
...판정...
/* 화면에 아무것도 출력하지 않고
   바로 system("cls") */
```

O 업그레이드 (After)




```
gotoxy(5, 14);                    /* 문제 아래에 */
printf("O 정답! (%.2f초)", check[order][i]);
...
Sleep(700);  clear_keys();        /* 0.7초 보여 주고 키 비우기 */
```

UPGRADE 11 / 21

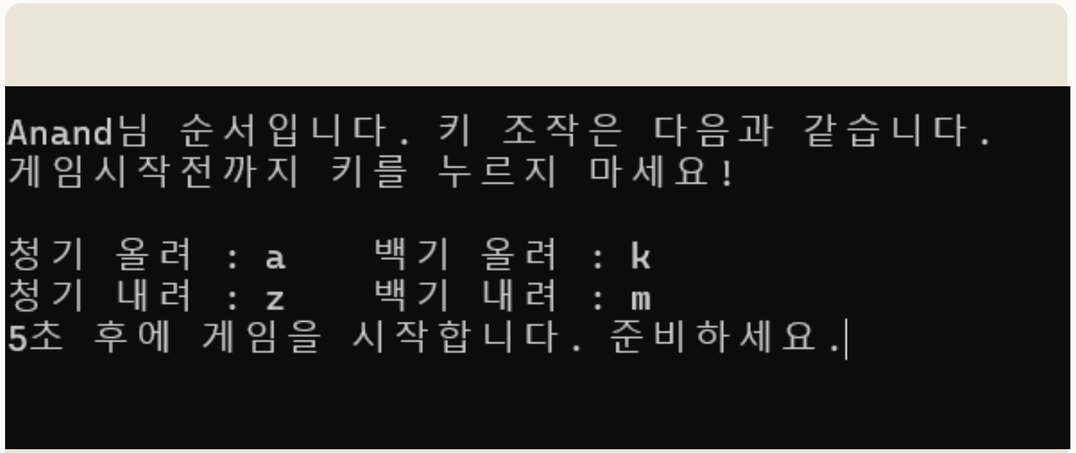
청기 · 백기 색 구분

변경

X 청기와 백기가 모두 같은 회색 글자

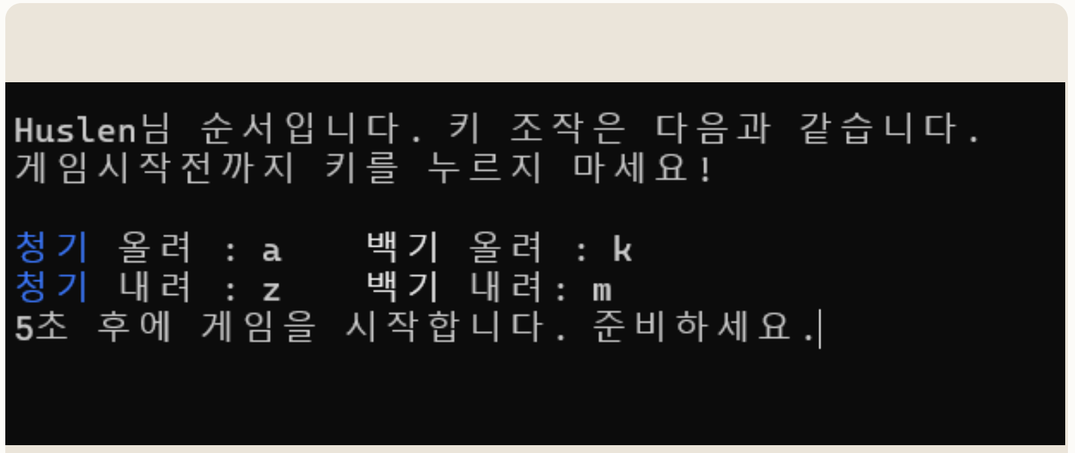
O 청기는 파란색, 백기는 흰색으로 구분

X 원본 (Before)




```
printf("청기 올려 : a    백기 올려 : k");
printf("%d: %s %s", i+1, flag[rnd[0]], ...);
```

O 업그레이드 (After)




```
void print_flag(int f) {
    if (f==0) { set_color(COLOR_BLUE);  printf("청기"); }
    else      { set_color(COLOR_WHITE); printf("백기"); }
    set_color(COLOR_GRAY);
}
```

UPGRADE 12 / 21

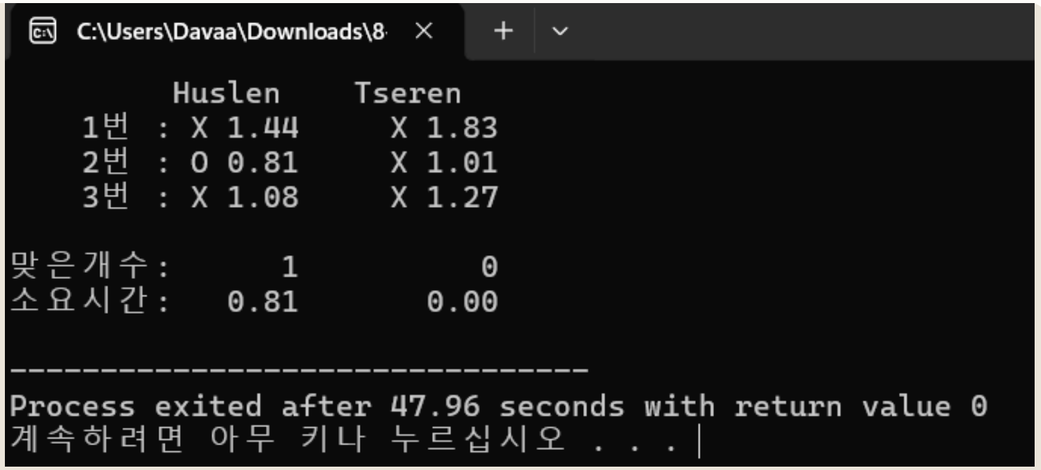
우승자 발표

추가

X 결과표만 나오고 누가 이겼는지는 직접 비교해야 한다

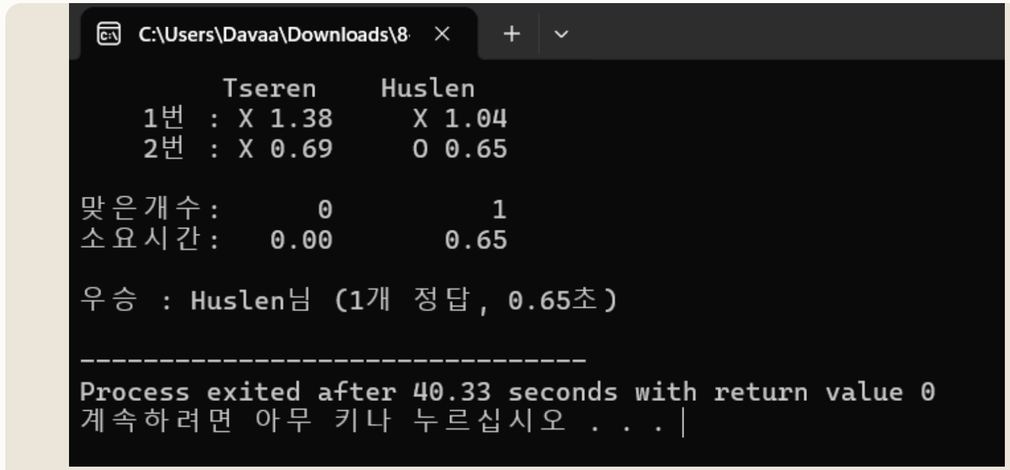
O 우승자를 자동으로 발표

X 원본 (Before)




```
printf("소요시간:");
for(i=0;i<user;i++)
    printf("%7.2f    ", min_time[order[i]]);
printf("\n");
}   /* 우승자 출력 없음 */
```

O 업그레이드 (After)




```
    best=order[0];                          /* 첫 사람을 일단 1등으로 */
    for(i=1;i<user;i++)
        if (result[order[i]]>result[best])      /* 더 많이 맞혔으면 교체 */
            best=order[i];
    printf("\n우승 : %s님 (%d개 정답, %.2f초)\n",
            name[best], result[best], total_time[best]);
}
```

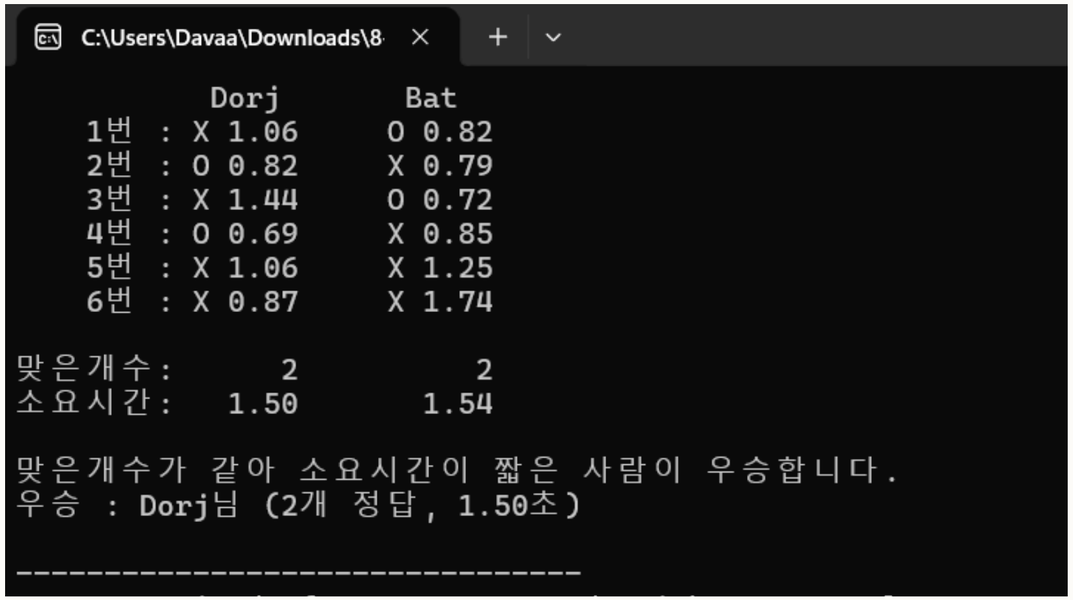
동점자 처리

추가

X 맞은 개수가 같으면 시간을 직접 비교해야 한다

O 동점이면 시간이 짧은 사람이 우승

O 업그레이드 · 동점 상황



판정 코드

```
    best=order[0];
for(i=1;i<user;i++)
{
    k=order[i];
    if (result[k]>result[best]) {          /* 더 많이 맞힘 → 교체 */
        best=k;
        tie=0;
    }
    else if (result[k]==result[best]) {    /* 맞은개수 같음 → 동점 */
        tie=1;
        if (total_time[k]<total_time[best])  /* 시간이 짧으면 교체 */
            best=k;
    }
}
if (tie)
    printf("\n맞은개수가 같아 소요시간이 짧은 사람이 우승합니다.");
printf("\n우승 : %s님 (%d개 정답, %.2f초)\n",
       name[best], result[best], total_time[best]);
}
```

UPGRADE 14 / 21

난이도(제한시간) 선택

변경

X 제한시간이 항상 1초로 정해져 있다

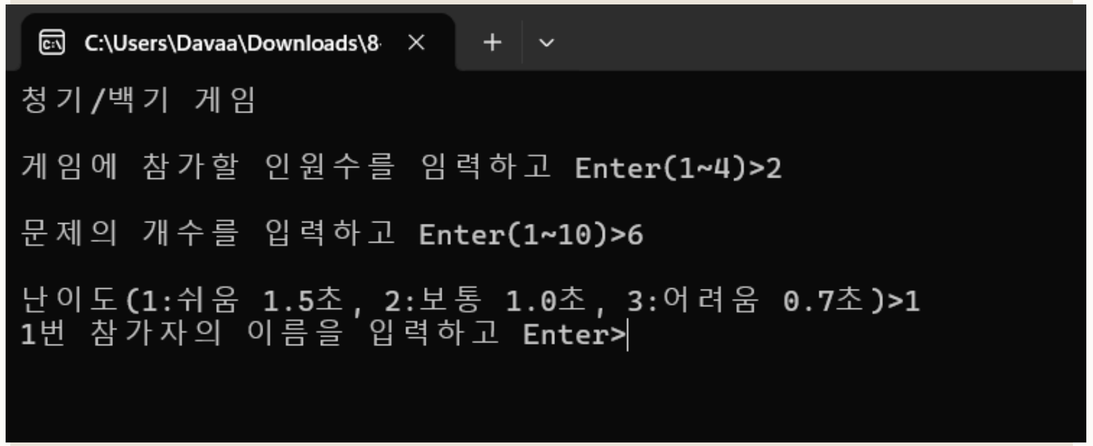
O 쉬움 1.5초 / 보통 1.0초 / 어려움 0.7초 선택

X 원본 (Before)

```
/* 1초가 코드에 고정 */
if (sol[order[i]][j] &&
    check[order[i]][j]<1)       /* 항상 1초 */
    printf("O");
```

모든 문제가 1초로 고정이라 초보자나 잘하는 사람이나 같은 조건으로만 할 수 있다.

O 업그레이드 (After)




```
level=input_number("난이도(1:쉬움 1.5초, ...)>", 1, 3);
if (level==1) return 1.5;         /* 쉬움 */
else if (level==2) return 1.0;    /* 보통 */
return 0.7;                       /* 어려움 */
...
check[order[i]][j]<limit[j]       /* 1 → 선택한 시간 */
```

UPGRADE 15 / 21

갈수록 짧아지는 제한시간

추가

X 모든 문제의 제한시간이 같아서 덜 긴장된다

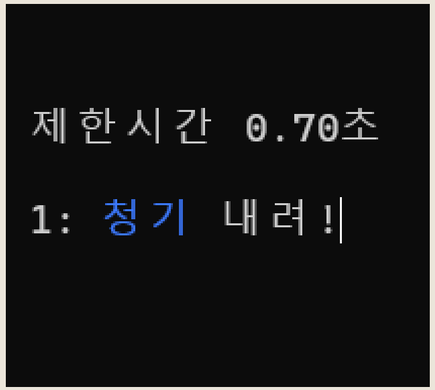
O 문제마다 5%씩 줄어든다 (1번 1.00초 → 5번 0.80초)

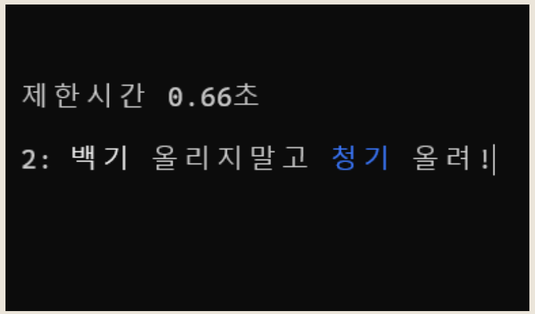
X 원본 (Before)

```
/* 1번 문제도 10번 문제도
   똑같이 1초 */
check[order[i]][j]<1
```

1번 문제나 마지막 문제나 똑같이 1초라서 갈수록 어려워지는 느낌이 없다.

O 업그레이드 (After) · 5번 문제는 0.80초






```
void make_limits(double base, double limit[], int quest) {
    for(i=0;i<quest;i++)
        limit[i]=base*(1.0-0.05*i);
}
```

UPGRADE 16 / 21

다시 하기

추가

X 한 판이 끝나면 프로그램이 바로 종료

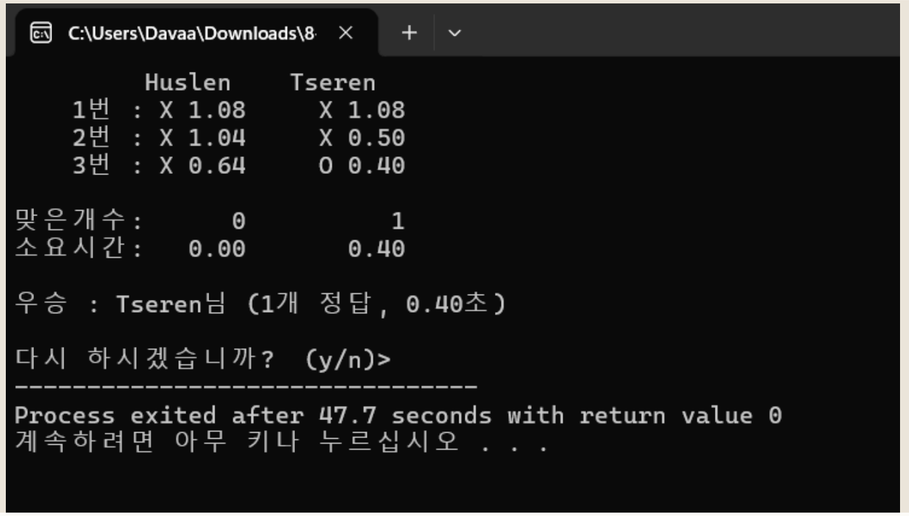
O y를 누르면 처음부터 다시 시작

X 원본 (Before)

```
display_result(...);
getch();
return 0;   /* 그대로 종료 */
```

결과를 본 뒤 아무 키나 누르면 프로그램이 끝나서, 다시 하려면 프로그램을 새로 실행해야 한다

O 업그레이드 (After) · 결과 뒤 다시 하기 질문




```
do {
    ...게임 전체...
    printf("\n다시 하시겠습니까? (y/n)>");
    again=tolower(getch());
} while(again=='y');
```

UPGRADE 17 / 21

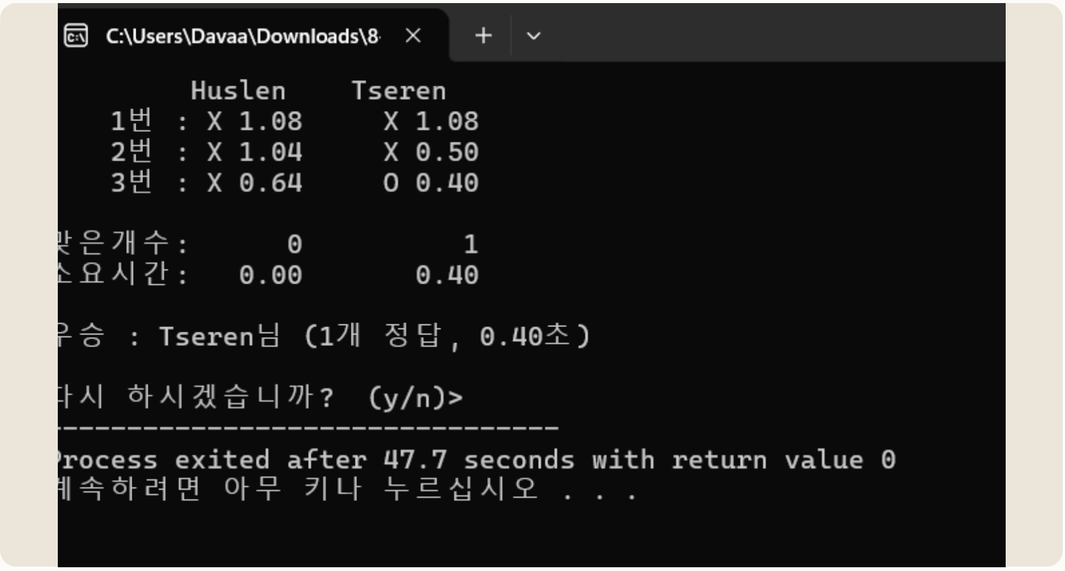
결과표 정렬

변경

X 이름 칸과 점수 칸의 폭이 달라 표가 비뚤어진다

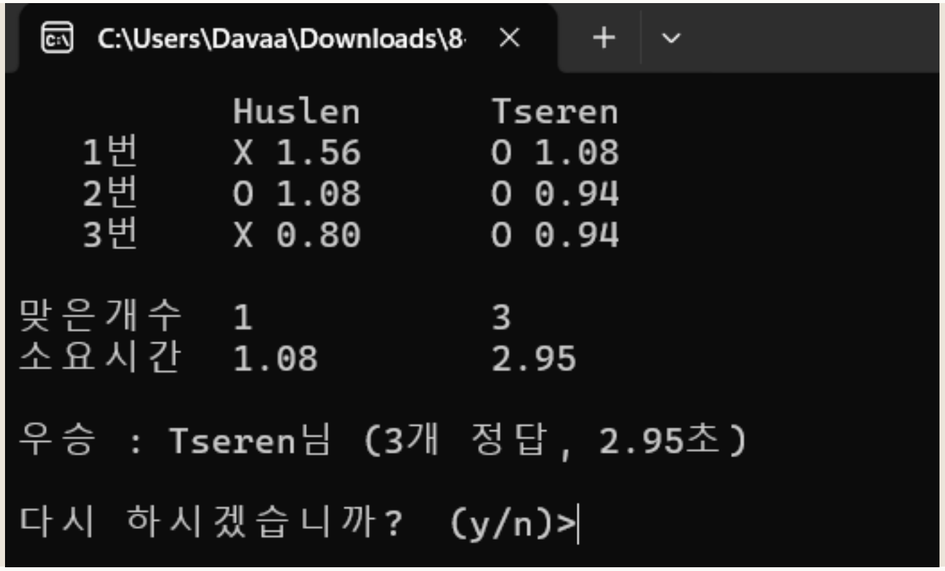
O 모든 칸을 12칸 폭으로 맞춘다

X 원본 (Before)




```
printf("%8s  ", name[order[i]]);   /* 10칸 */
printf("    %d번 : ", j+1);
printf("O"); printf("%5.2f     ", ...);  /* 11칸 */
```

O 업그레이드 (After)




```
printf("%-12s", name[order[i]]);   /* 12칸 */
printf("  %2d번    ", j+1);
printf("O "); printf("%-10.2f", ...);  /* 12칸 */
```

UPGRADE 18 / 21

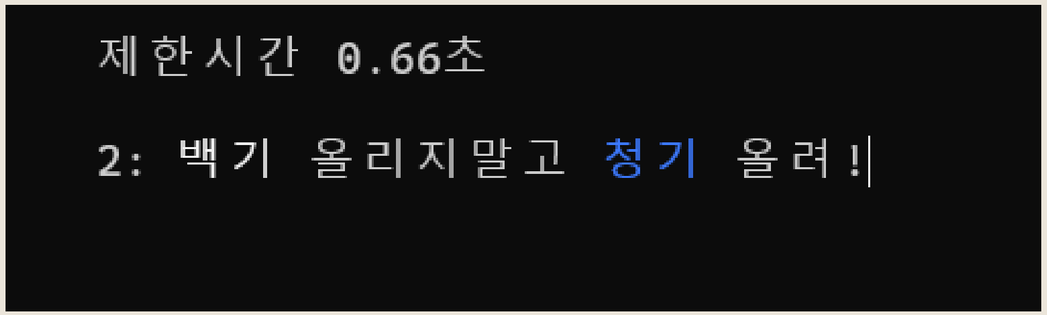
깃발 그림 보여주기

추가

X 글자로만 명령이 나온다

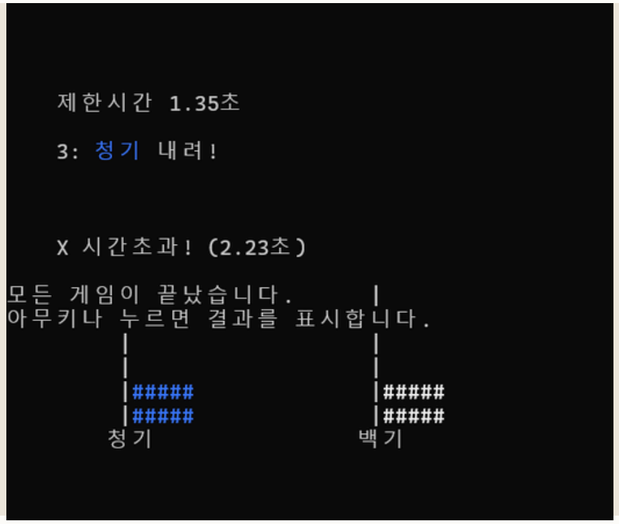
O 누른 키에 따라 깃발이 올라가고 내려가는 그림

X 원본 (Before)



draw_flags()가 gotoxy로 깃대(|)와 깃발(#####)을 그린다.
올리면 위쪽 2줄, 내리면 아래쪽 2줄에 색을 칠한다. 오른쪽 화면은
k(백기 올려)를 누른 직후.

O 업그레이드 (After)




```
if (key=='a') blue_up=1;
else if (key=='z') blue_up=0;
else if (key=='k') white_up=1;
else if (key=='m') white_up=0;
draw_flags(blue_up, white_up);
```

UPGRADE 19 / 21

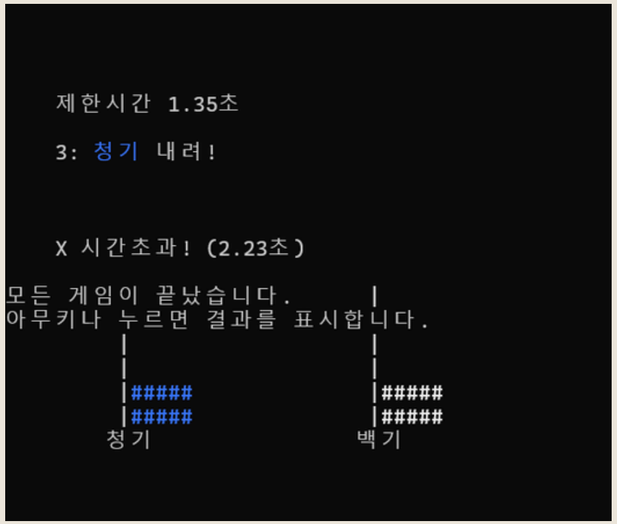
제한시간이 지나면 자동으로 넘어가기

추가

X 키를 안 누르면 제한시간이 지나도 끝없이 기다린다

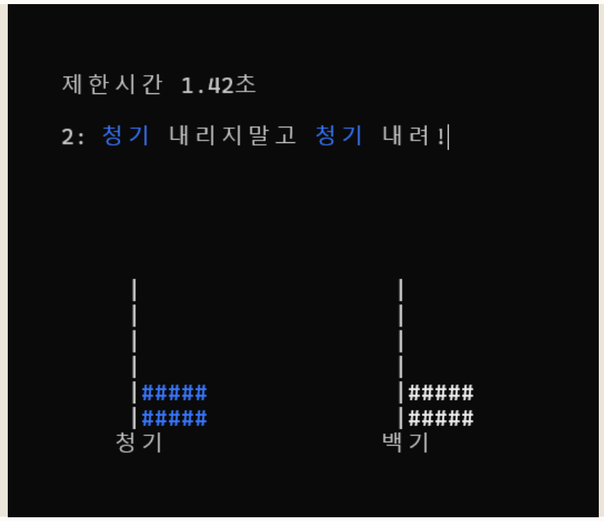
O 시간이 지나면 '시간초과'로 하고 다음 문제로

X 원본 (Before)



키를 안 누르고 5초를 기다려도 문제가 그대로 멈춰 있다.

O 업그레이드 (After)



kbhit()은 기다리지 않고 키가 눌렸는지만 확인한다. 그래서 시간도 같이 검사할 수 있다.

UPGRADE 20 / 21

최고 기록 저장

추가

X 프로그램을 끄면 기록이 모두 사라진다

O best.txt에 저장하고 다음에 켜면 보여 준다

X 원본 (Before)

```
청기/백기 게임

게임에 참가할 인원수를 입력하고 Enter(1~4)>1

문제의 개수를 입력하고 Enter(1~10)>1

난이도(1:쉬움 1.5초, 2:보통 1.0초, 3:어려움 0.7초)>1
1번 참가자의 이름을 입력하고 Enter>Tseren
만약 한글을 사용하였다면 영문키로 바꾸십시오.
아무키나 누르시오.

게임의 순서는 다음과 같습니다.
1번 : Tseren
아무키나 누르시오.
```

```
FILE *fp=fopen("best.txt", "w");    /* 쓰기 */
fprintf(fp, "%s %d %.2f\n", name, correct, time);
fclose(fp);
```

O 업그레이드 (After)

```
C:\Users\Davaa\Downloads\8
청기/백기 게임

아직 최고 기록이 없습니다.

게임에 참가할 인원수를 입력하고 Enter(1~4)>2

문제의 개수를 입력하고 Enter(1~10)>3

난이도(1:쉬움 1.5초, 2:보통 1.0초, 3:어려움 0.7초)>1
1번 참가자의 이름을 입력하고 Enter>Huslen
2번 참가자의 이름을 입력하고 Enter>Tserentogtokh
만약 한글을 사용하였다면 영문키로 바꾸십시오.
아무키나 누르시오.

게임의 순서는 다음과 같습니다.
1번 : Tseren
2번 : Huslen
아무키나 누르시오.
```

```
FILE *fp=fopen("best.txt", "r");    /* 읽기 */
if (fp==NULL) return 0;             /* 파일 없음 = 기록 없음 */
fscanf(fp, "%6s %d %lf", name, correct, time);
```

기준은 13번과 같다: 많이 맞힌 사람 → 같으면 빠른 사람.

UPGRADE 21 / 21

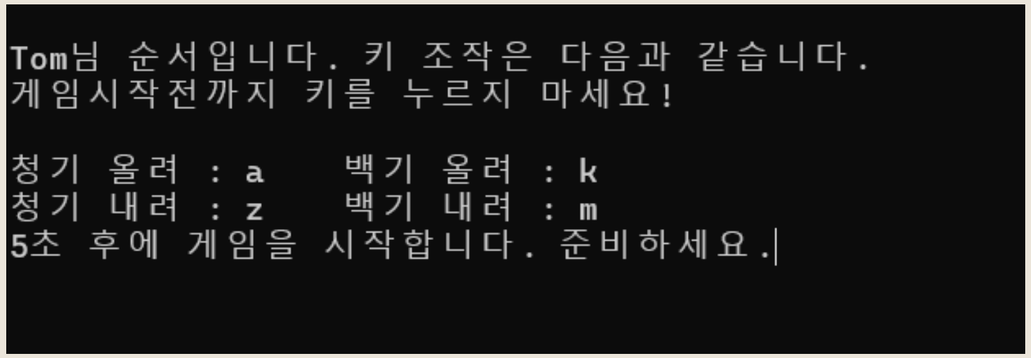
3 · 2 · 1 시작 알림

변경

X '5초 후 시작' 글자만 있어서 언제 시작하는지 모른다

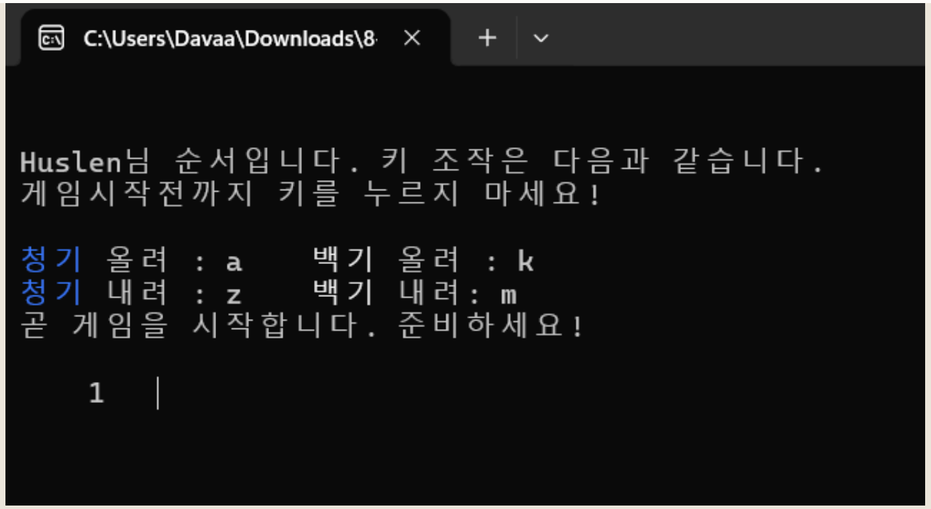
O 3 → 2 → 1 → 시작! 을 소리와 함께 보여 준다

X 원본 (Before)




```
printf("5초 후에 게임을 시작합니다. 준비하세요.");
Sleep(5000);                /* 5초 동안 아무 변화 없음 */
```

O 업그레이드 (After)

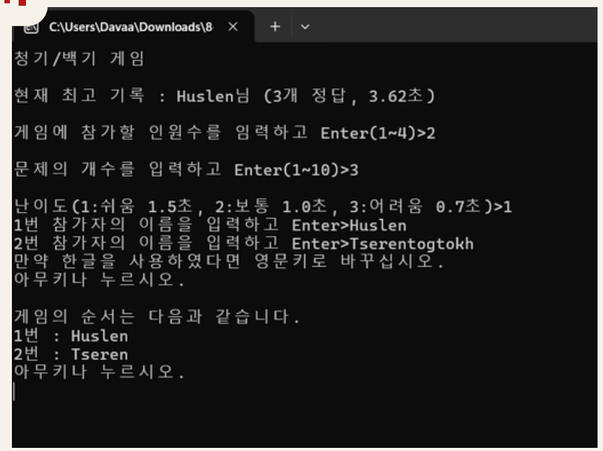



```
for(i=3;i>=1;i--) {
    gotoxy(5, 10);
    printf("%d   ", i);      /* 같은 자리에 숫자 덮어쓰기 */
    Beep(600, 150);
    Sleep(1000);
}
printf("시작!");
```

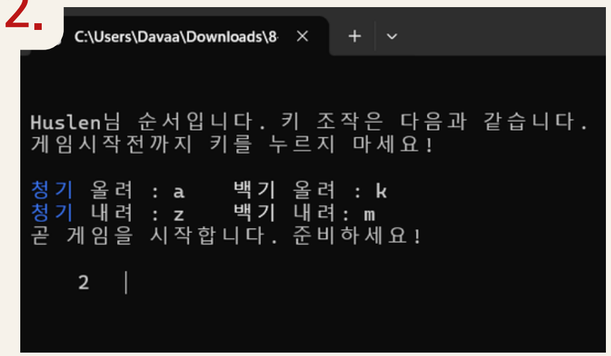
Result

최종 실행 화면
21개 업그레이드를 모두
적용한 실행 결과

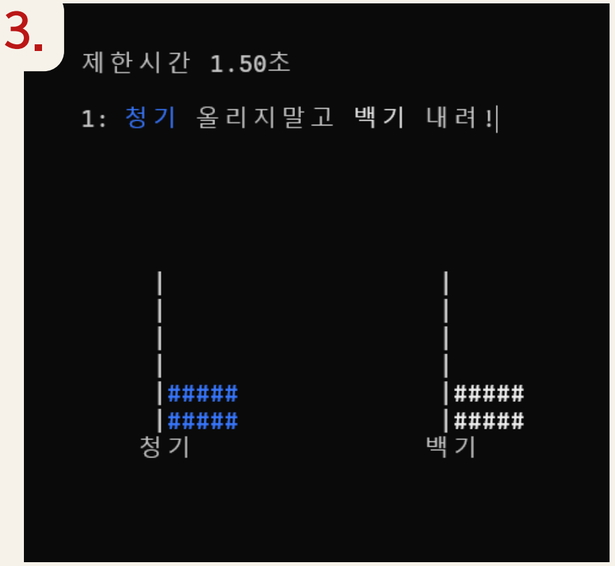
1.



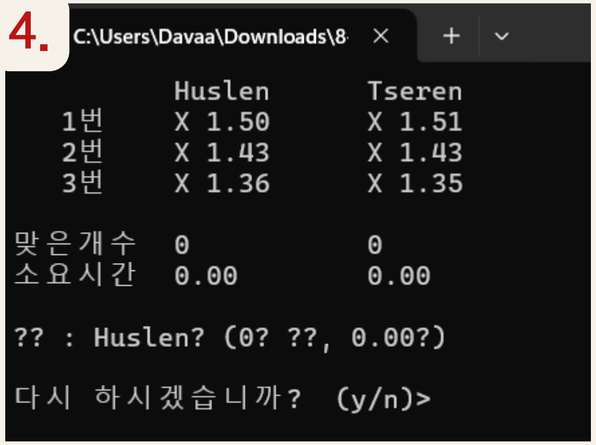
2.



3.



4.



최종 실행 화면 · 요약표

- **추가 11개**
- **변경 9개**
- **삭제 1개**

01 인원수 · 문제 수 입력 범위 검사	12 우승자 발표
02 이름 길이 제한	13 동점자 처리
03 미리 누른 키 무시	14 난이도(제한시간) 선택
04 대 · 소문자 모두 인식	15 갈수록 짧아지는 제한시간
05 시간 계산 이름 바꾸기	16 다시 하기
06 필요 없는 & 지우기 · 경고 없애기	17 결과표 정렬
07 변수 이름 바로잡기	18 깃발 그림 보여주기
08 공정한 순서 섞기 (Fisher-Yates)	19 제한시간이 지나면 자동으로 넘어가기
09 정답 / 오답 효과음	20 최고 기록 저장
10 문제마다 바로 결과 보여주기	21 3 · 2 · 1 시작 알림
11 청기 · 백기 색 구분	

결론 / 느낀 점

01 작은 게임도 기본이 중요하다

177줄짜리 게임에서도 배열 범위, 입력 길이, 입력 버퍼처럼 눈에 안 보이는 곳에서 버그가 나왔다.

02 버그는 화면으로 증명한다

같은 입력으로 원본과 업그레이드를 실행해 차이를 직접 보여 주니 설명이 쉬워졌다.

21/30

개 업그레이드

Thank you

감사합니다